

Análisis de datos

Carrera de Especialización en
Inteligencia Artificial
BI 2026



Programa de la materia (detallado)

1

Introducción al análisis de datos

4/3

Flujo de trabajo típico. Aplicaciones. Configuración del entorno. Introducción a las principales bibliotecas. Introducción al EDA.

2

Análisis exploratorio de datos (EDA)

11/3

EDA. Estadística robusta. Visualización de variables numéricas y categóricas. Relación entre variables. Data cleaning y data wrangling. Identificación de valores faltantes y outliers.

3

Taller práctico - parte 1

18/3

Presentación de grupos de trabajo final. Práctica en salas virtuales: EDA y visualización inicial de un dataset. Puesta en común.

4

Preprocesamiento y limpieza de datos

25/3

Flujo de preparación de datos. Prevención de data leakage. Tratamiento de datos faltantes y outliers. Cardinalidad. Codificación. Discretización.



5

Procesamiento (continuación)

1/4

Normalización y estandarización. Técnicas avanzadas para el tratamiento de datos faltantes y outliers. Desbalance. Creación de nuevos features.

6

Reducción de dimensionalidad

8/4

Análisis estadístico inferencial. Técnicas para selección y extracción de features.
Bonus: EDA de datos no estructurados.

7

Taller práctico - parte 2

15/4

Práctica en salas virtuales: preprocesamiento y feature engineering de un dataset. Puesta en común.
Bonus: Herramientas para automatizar el EDA.

8

Exposición final

22/4 y
25/4

Exposición de trabajos finales (dos sesiones)

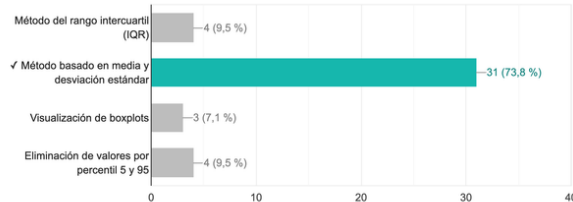
ACTIVIDAD INDIVIDUAL 3

Estadística clásica

- Usa medidas como media y varianza.
- Supone que los datos siguen distribuciones conocidas (ej: normal).
- Es sensible a valores atípicos.

3. ¿Cuál de los siguientes es un método clásico para detectar outliers?

31 de 42 respuestas correctas

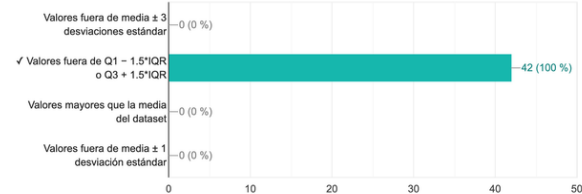


Estadística robusta

- Usa medidas como mediana o IQR.
- No depende de supuestos de distribución.
- Es resistente a valores atípicos.

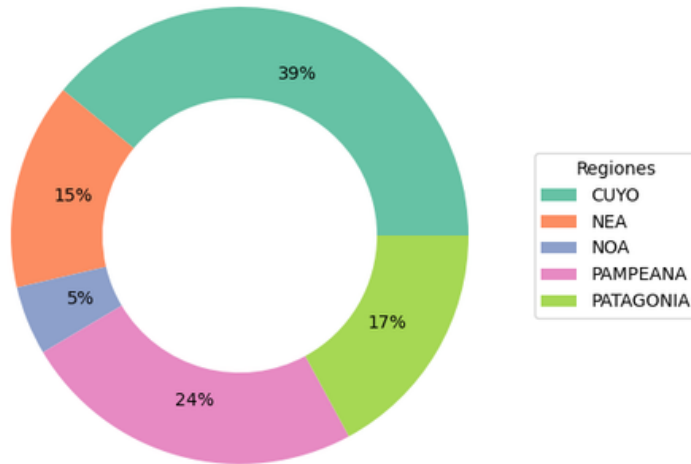
4. ¿Cuál de los siguientes criterios corresponde a un método robusto para detectar outliers?

42 de 42 respuestas correctas

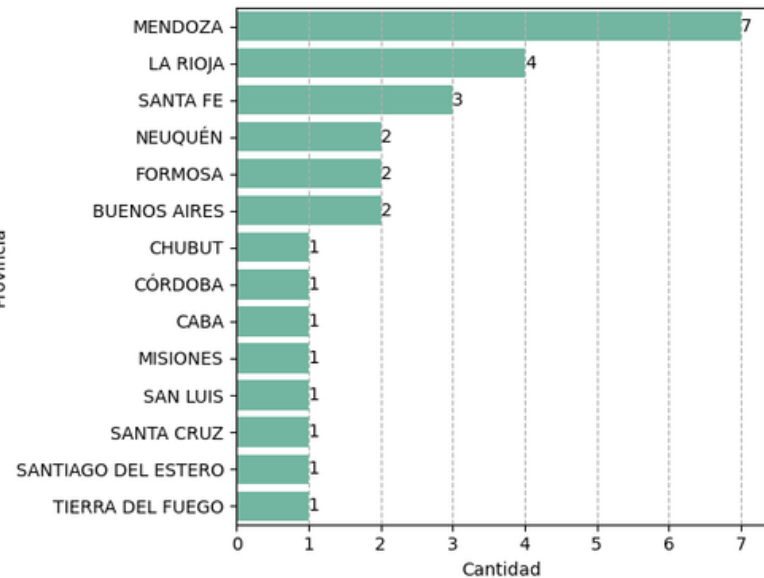


ACTIVIDAD INDIVIDUAL 3

Regiones elegidas



Provincias elegidas



ACTIVIDAD INDIVIDUAL 3

Caracterización climática

Elegí una provincia de la región y describí brevemente su clima utilizando al menos tres variables del dataset (temperatura, precipitación, humedad, nubosidad o viento).

```
columnas = ['prec_sup_1mm', 'humedad', 'nubosidad', 'prec_mm', 'temp']  
smn_historico[smn_historico['provincia']=='MENDOZA'][columnas].describe()
```

	prec_sup_1mm	humedad	nubosidad	prec_mm	temp
count	60.000000	60.000000	60.000000	60.000000	60.000000
mean	2.933333	56.086667	2.993333	24.185000	15.586667
std	1.215809	8.102447	0.397393	13.560421	6.288932
min	0.600000	40.000000	2.300000	3.100000	3.800000
25%	2.000000	49.925000	2.700000	12.025000	10.325000
50%	2.950000	54.100000	2.950000	23.850000	15.950000
75%	3.800000	63.275000	3.200000	32.675000	21.050000
max	5.400000	70.400000	4.000000	54.200000	25.900000

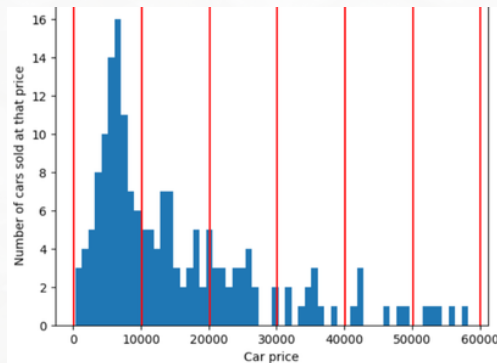


Ubicada en la región de Cuyo, la provincia de Mendoza presenta un clima caracterizado por una temperatura promedio de 15.59°C. Sus registros meteorológicos muestran precipitaciones moderadas que rondan los 24.19 mm, acompañadas de una humedad promedio del 56.09%, lo que define el clima característico de esta zona.

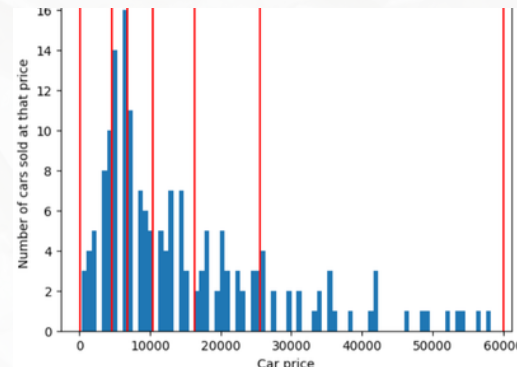
Discretización

- La discretización convierte **variables continuas** en variables discretas o **categorías**.
- Se usa para mejorar la interpretación o el rendimiento de un modelo.

Métodos de discretización

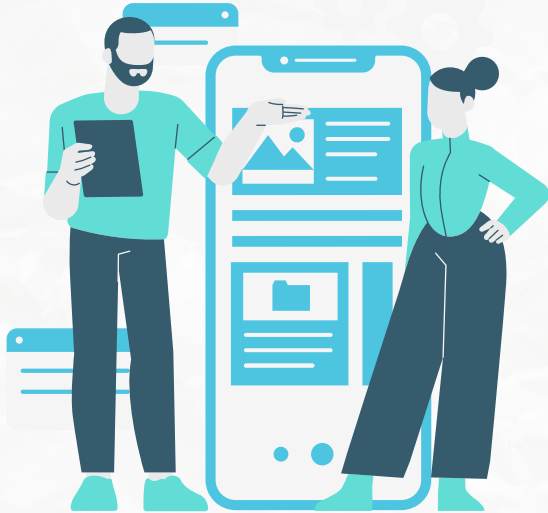


- Por intervalos de igual rango de valores.



- Por puntos de corte (cuantiles)

¿Para qué discretizamos?



- Para reducir la cantidad de datos y la complejidad computacional.
- Para disminuir el “ruido”: muchas veces los datos crudos incluyen fluctuaciones aleatorias o irrelevantes que contribuyen al overfitting.
- Para adaptar la variable target (convertir una variable continua a discreta) en un problema de clasificación.
- Para mejorar el entendimiento o la interpretación de un problema (por ejemplo, convertir edades de clientes en segmentos objetivo: joven, adulto, adulto mayor).
- Como herramienta para “contener” el efecto de los outliers.

Ejemplos prácticos en Jupyter



RESUMEN de métodos de discretización

Método	Descripción	Pros y contras
Intervalos iguales	Rangos de igual amplitud.	Simple. No sirve si los datos son asimétricos.
Por puntos de corte	Basados en la distribución de los datos. Cada intervalo tiene el mismo número de observaciones.	Funciona bien para datos asimétricos.
K-means	Agrupar los datos en clusters con el algoritmo k-means	Captura agrupamientos naturales. Para datasets grandes requiere mucho cómputo.
Ancho fijo	Definidos por el usuario basados en el conocimiento del dominio.	Control total, pero requiere conocimiento de los datos y su distribución.

Recap de lo visto en la clase 4



Flujo de preparación de datos. Split.

Dataset original

Estrategia de split

x1	x2	...	target



x1	x2	...	target



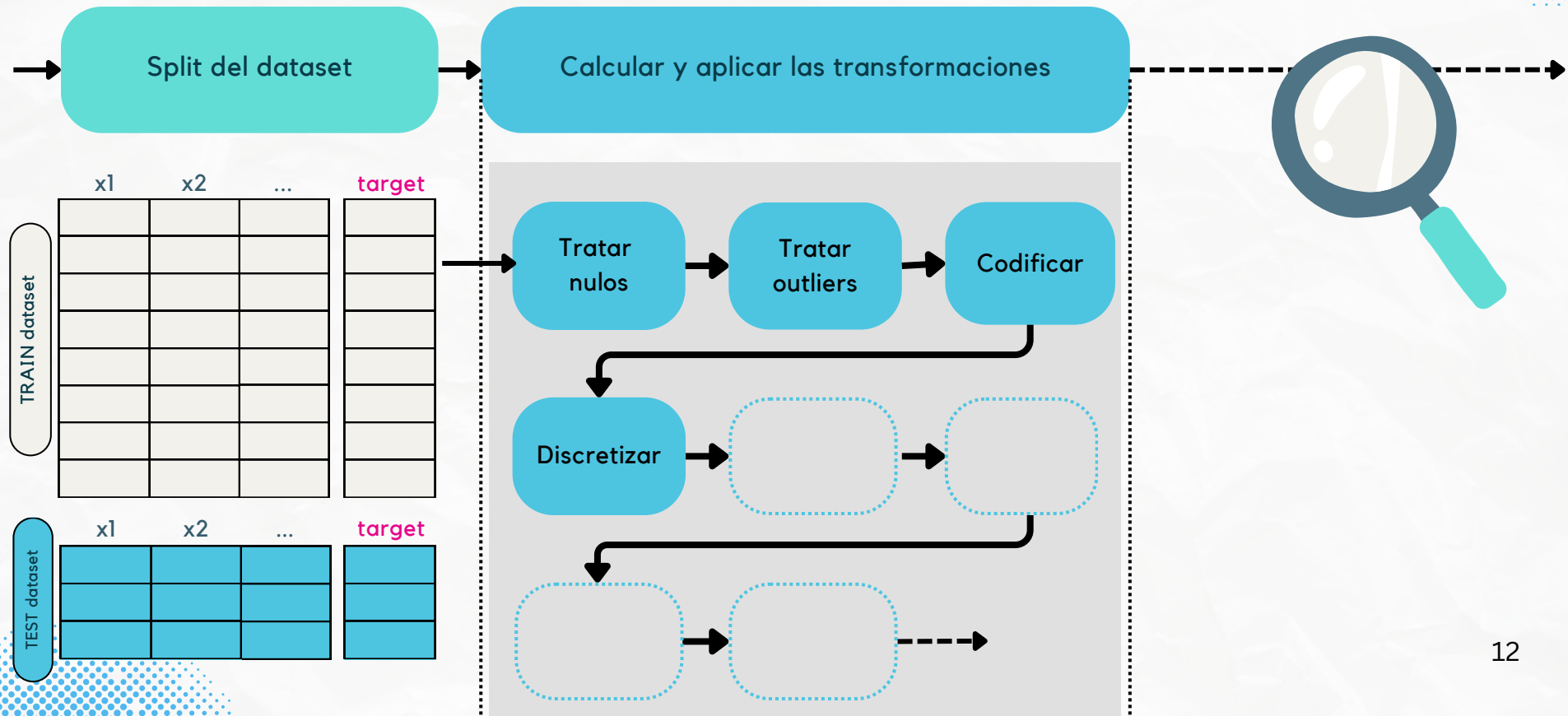
x1	x2	...	target

TRAIN
dataset

x1	x2	...	target

TEST
dataset

Flujo de preparación de datos



Normalización y estandarización

- La normalización y la estandarización son dos técnicas para preparar las variables con el objetivo de mejorar el rendimiento de los modelos.

- **Normalización.** Se escalan las variables numéricas para que sus valores estén dentro de un rango específico, generalmente entre 0 y 1.

- Útil cuando se usa un algoritmo que depende de la distancia (ej. KNN)

$$X_{\text{normalizado}} = \frac{X - \min(X)}{\max(X) - \min(X)}$$

- **Estandarización** (o Z-score). Se transforman los datos de modo que tengan una media igual a 0 y una desviación estándar igual a 1.

- Adecuada cuando los datos ya tienen una distribución normal.
- Útil para algoritmos que asumen distribución normal (Regresión lineal, SVM)

$$X_{\text{estandarizado}} = \frac{X - \mu}{\sigma}$$

Actividad



Escalamiento después de codificación

- La decisión de escalar las variables categóricas dependerá del algoritmo de ML a usar y del mecanismo de codificación empleado.
- Excepto algunos casos, la mayoría de las técnicas pueden producir rangos grandes de números y es una buena práctica escalar siempre después de codificar (especialmente si se utilizarán algoritmos basados en distancia).
- En general, no se escalan los features binarios (one-hot encoded) ya que esto no aporta ningún valor (y en algunos casos puede incluso ser contraproducente).
- Tampoco es aconsejable modificar features codificados con cyclic encoding, ya que se "rompe" el efecto cíclico.



Técnicas avanzadas para el tratamiento de datos faltantes



Habíamos visto métodos simples como:

- Eliminación de filas (observaciones) y columnas (variables).
- Imputación por métodos univariados (ejemplo: imputar por la media)

Además, podemos aplicar imputación por métodos *multivariados* (a partir de valores de otras columnas):

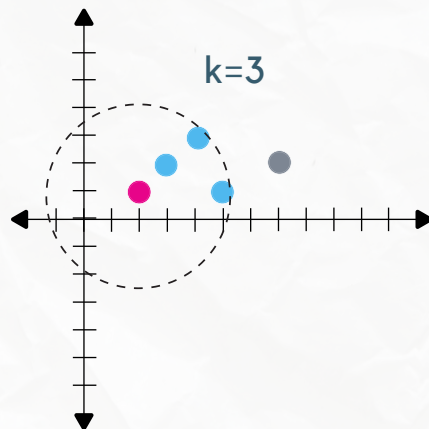
- algoritmos de ML (por ej., regresión, árboles de decisión, KNN).
- modelos de imputación múltiple (ej., MICE)

Métodos multivariados - KNN

KNN (K-Nearest Neighbors):
busca los "k" vecinos que se
parecen más a la fila del dato
faltante. Este se calcula luego a
partir de esos vecinos.

Atención:

- Se recomienda normalizar los datos antes de usar este algoritmo porque es sensible a la escala.
- Puede ser ineficiente en grandes volúmenes de datos.



v1	v2	v3
4	3	2
7	2	2
2	1	NaN
5	1	4
3	2	3

3

KNNImputer

Gallery examples: Imputing missing values before building an estimator
Highlights for scikit-learn 0.22



Métodos multivariados - MICE

- MICE (*Multiple Imputation by Chained Equations*) es una técnica iterativa avanzada de imputación múltiple (muchas variables a la vez).
- Una a la vez, cada columna con faltantes se define como variable target. Con el resto de los features se ajusta un modelo, que luego se usa para predecir e imputar cada NaN de la columna en cuestión.
- Se realizan N iteraciones (número de imputaciones del dataset completo que se quieren hacer).



IterativeImputer

(En modo experimental)

Gallery examples: Imputing missing values with variants of IterativeImputer Imputing missing values before building an estimator



scikit-learn

Mice imputation (statsmodels)

Dataset con nulos

A	B	C

1

Imputación simple

A	B	C
		Media C
Media A		
	Media B	



Ojo! la notación $A \sim B + C$ se llama "fórmula estilo Patsy" y significa que A se predice a partir de B y de C (no es una suma!).

MICE paso a paso

2

Ajustamos un modelo $A \sim B + C$

A	B	C
		Media C
	Media B	

Calculamos los nulos A_i con el modelo y los B_i, C_i

A	B	C
		Media C
Pred A_i II	B_i	C_i
	Media B	

3

Ajustamos un modelo $B \sim A + C$

A	B	C
		Media C
Pred A II		

Calculamos los nulos B_i con el modelo y los A_i, C_i

A	B	C
		Media C
Pred A II		
A_i	Pred B_i II	C_i

4

Ajustamos un modelo $C \sim A + B$

A	B	C
Pred A II		
	Pred B II	

Calculamos los nulos C_i con el modelo y los A_i, B_i

A	B	C
A_i	B_i	Pred C_i II
Pred A II		
	Pred B II	

5

Iteración 1 completa!

A	B	C
		Pred C II
Pred A II		
	Pred B II	

6

Continuar con la próxima vuelta (repetir desde el paso 2 al 5) hasta completar el nro. de iteraciones.

Estrategias avanzadas para tratamiento de faltantes

- Si son MAR con patrones claros y el porcentaje es moderado a alto (20-50%):
 - Imputación con modelos de ML para variables críticas.
 - Algoritmos como *Random Forest Imputer* o KNN-imputer si los patrones son no lineales.
- Si se sospecha MAR con patrones complejos (los faltantes dependen de muchas variables):
 - *Multiple Imputation by Chained Equations* (MICE).
- En el caso de MNAR, se requiere modelar e implementar explícitamente el mecanismo de los faltantes con soluciones *custom*.

Ejemplos prácticos en Jupyter



Técnicas avanzadas para el tratamiento de outliers



Habíamos visto métodos simples como:

- Eliminación de filas (observaciones)
- Imputación por métodos univariados (ejemplo: imputar por la media, *capping*)
- Minimizar el impacto con transformaciones (ejemplo: logaritmo)
- Segmentación

Además, podemos aplicar imputación por métodos *multivariados* (a partir de valores de otras columnas):

- algoritmos de ML (por ej. KNN).

Ejemplos prácticos en Jupyter



Creación de nuevos features

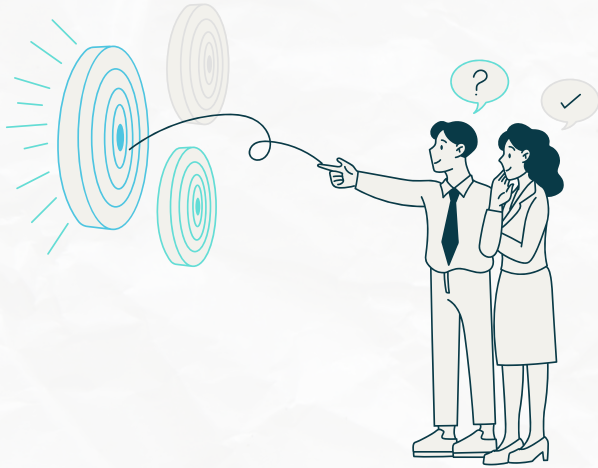


- Se trata de generar nuevas variables a partir de otras existentes.
- El objetivo es captar patrones o relaciones ocultas y mejorar la interpretación.

Ejemplo:

- $IMC = \text{peso} / (\text{altura})^2$
- Reemplazar las variables peso y altura por la métrica IMC, que está más relacionada con la salud que las originales.

(Algunas) técnicas para crear features



- Matemáticas: A/B , $A*B$, $A-B$, etc.
- Agregados: promedios, totales por grupo, sumas, etc.
- Temporales: extraer día, hora, mes, de un timestamp.
- Texto: TF-IDF (importancia de una palabra), generación de embeddings.

Ejemplos comunes

- Fecha de ingreso → Antigüedad/categoría (senior, semi-senior, etc.)/ tiempo desde el último evento (por ej promoción).
- Fecha de nacimiento → Edad.
- Timestamp → Hora del día/Es día hábil?/fin de semana?/feriado?/llovió ese día?/Temporada?
- Factores de riesgo (tabaquismo, hipertensión, diabetes, etc.) → Conteo /Nivel de riesgo ponderado/ocurrencia simultánea.
- Valores faltantes → Flag / conteo de faltantes por fila / faltante por dominio (ej: financiero, demográfico, etc.)
- Número de cel → localización geográfica a través del código de área.
- Sueldo/provincia/profesión → Salario promedio por provincia por profesión (ojo! media global si hay combinaciones muy raras)
- Transacciones/usuarios → número de transacciones por usuario.
- *Feedback* de un cliente → *Sentiment score* (positivo, neutral, negativo)
- Descripción de un producto → Palabras clave (descuento, oferta, premium, etc.)

Criterios para crear features



- Deben que ser relevantes para el objetivo.
- Tienen que capturar relaciones importantes o transformaciones de variables guiadas por conocimiento del dominio o análisis.
- Tienen que complementar o superar lo que aportan las variables originales.
- Creados teniendo en cuenta los modelos que los consumirán.
- Hay un balance entre costo computacional e impacto en performance.

Ejemplos de casos de uso:

- Dataset con pocas filas.
- Patrones no lineales.
- El conocimiento del dominio lo pide (deuda/ingreso, nro. de ítems por compra, etc).

Tip: conocer los modelos

- Agregar features con relaciones complejas (ej., IMC) cuando se trabaja con modelos lineales.
- Considerar agregar ratios, que son difíciles de aprender para casi todos los modelos.
- Los árboles pueden aprender casi cualquier combinación de features, pero cuando hay alguna combinación específica que nos interesa conviene crearla explícitamente (especialmente si hay pocos datos).
- Los features de tipo *aggregation* son especialmente útiles para modelos de tipo árbol porque estos no tienen la capacidad natural de combinar información de muchos features a la vez.

Desbalance

Se produce cuando las clases (o etiquetas) que el modelo debe predecir no están representadas de manera proporcional en el dataset.

Causas:

- Sesgo al momento de recolectar los datos.
- Naturaleza del dominio en cuestión (causa más frecuente).

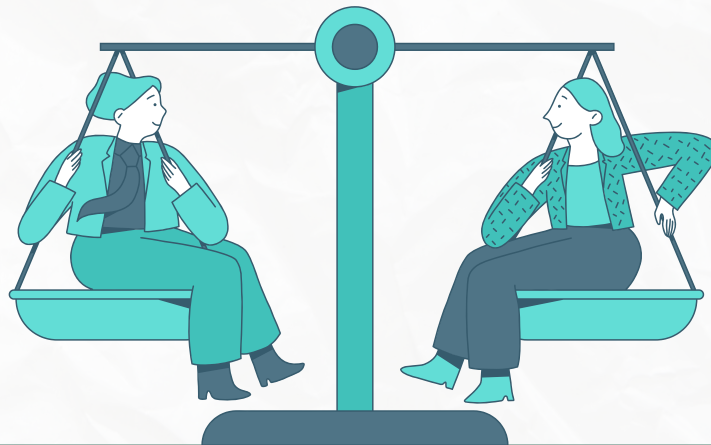


Desbalance

¿Por qué es un problema?

- Los modelos de ML tienden a aprender mejor de las clases más fuertes (mayoritarias) y tener un mal desempeño para las clases menos frecuentes (minoritarias).
- En consecuencia las métricas como precision, recall, F1-score y AUC-ROC, pueden ser engañosas.

Ejemplo: un modelo que siempre predice la clase mayoritaria tiene alta accuracy pero bajo recall en las clases minoritarias.



Técnicas para detectar desbalance

Técnica	Explicación
Análisis de la distribución	Análisis visual (gráfico de barras, pie chart) o cálculo de las proporciones de cada clase
Matriz de confusión	Según cómo se clasificaron las instancias puede revelarse un desbalanceo de clases.
Índice de Gini $G = 1 - \sum p_i^2$	Mide la pureza de los datos. Indirectamente puede reflejar si hay un desbalance entre clases. ~0.5 indica un dataset balanceado.
Entropía de Shannon $H = - \sum_{i=1}^n p_i \log_2(p_i)$	Mide la incertidumbre de los datos. Mientras más alta la entropía, mayor la incertidumbre, más balanceados los datos.

Técnicas para el tratamiento del desbalance

Técnica para balancear	Explicación
<i>Oversampling</i> (sobremuestreo)	Duplicar o recrear en forma sintética las muestras de la clase minoritaria: Ej. SMOTE (<i>Synthetic Minority Over-sampling Technique</i>) o IA generativa.
<i>Undersampling</i> (submuestreo)	Remover muestras de la clase mayoritaria.

Técnica para mitigar	Explicación
<i>Weighted Random Sampler</i> (muestreo aleatorio ponderado)	Utiliza muestreo con reemplazo proporcional a los pesos (generalmente inversos a la frecuencia de la clase). Esto simula un dataset equilibrado sin alterar los datos originales.
<i>Weighted Loss Functions</i> (Funciones de pérdida ponderadas)	Asigna pesos a las clases en la función de pérdida (ej. en <i>Cross-Entropy Loss</i>). Esto hace que el optimizador (ej. Adam) se enfoque más en minimizar errores en clases minoritarias.
Técnicas de ensamble	Algoritmos como <i>Balanced Random Forest</i> y AdaBoost tienen mecanismos internos para manejar el desbalance.

En desbalances extremos (1:1000), considerar recolectar más datos.

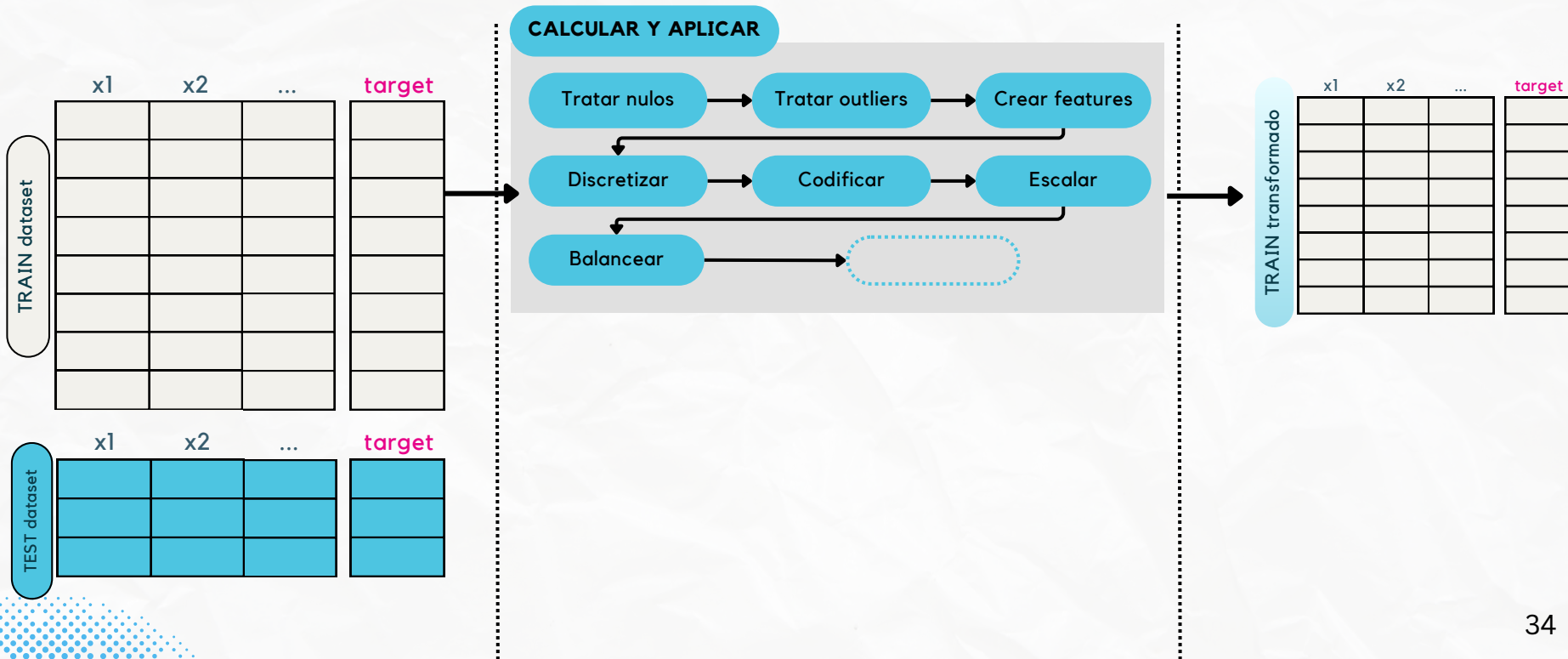
Ejemplo práctico



Flujo de preparación de datos

Split del dataset

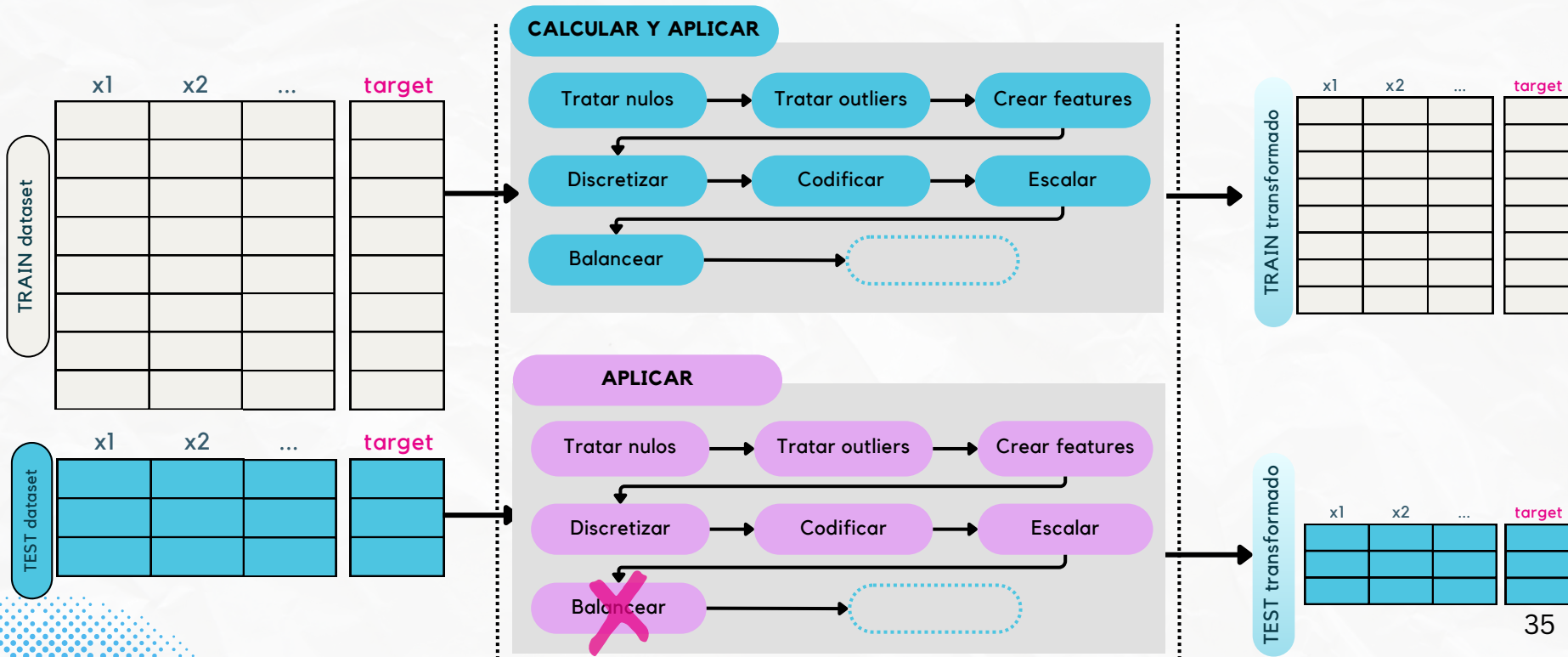
Calcular y aplicar las transformaciones



Flujo de preparación de datos

Split del dataset

Calcular y aplicar las transformaciones



Calcular y aplicar transformaciones

Transformación	Ejemplo	Calcular y aplicar en TRAIN	Aplicar en TEST
Tratar nulos	Imputar por la media	Calcular la media: 28 Imputar todos los nulos con 28.	Imputar todos los nulos con 28.
Tratar outliers	Clipping	Determinar outliers: > 65 Imputar todos los outliers con 65.	Imputar con 65 todos los valores mayores a 65.
Crear features (el cálculo no depende de otras observaciones)	IMC	$IMC = peso / (altura)^2$	$IMC = peso / (altura)^2$
Crear features (el cálculo depende de otras observaciones)	edad promedio por provincia	Calcular los valores de edad promedio por provincia. Ejemplo: Neuquén: 45. A las observaciones de Neuquén se les asigna 45.	Se asignan los valores según el cálculo hecho en entrenamiento. A las observaciones de Neuquén se les asigna 45.

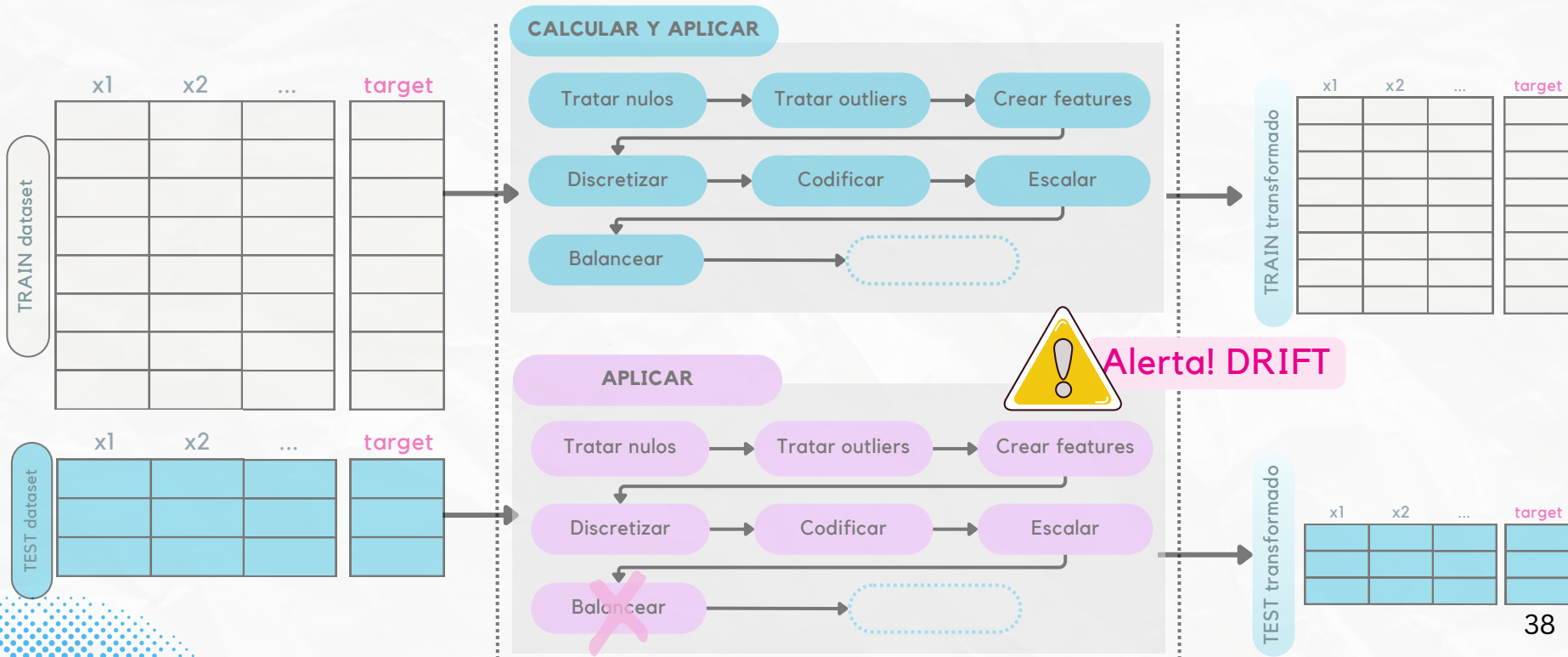
Calcular y aplicar transformaciones

Transformación	Ejemplo	Calcular y aplicar en TRAIN	Aplicar en TEST
Discretizar	Por rangos fijos	Calcular los rangos en base a la distribución de la variable: 10-20, 21,-30 Asignar la categoría: A (si el valor está entre 10 y 20) B (si está entre 21 y 30)	Asignar la categoría: A (si el valor está entre 10 y 20) B (si está entre 21 y 30)
Codificar	One hot encoding	Calcular las categorías: rojo, azul, verde. Crear tres columnas: R, A, V. Codificar rojo como 100, azul como 010 y verde como 001.	Codificar rojo como 100, azul como 010 y verde como 001.
Escalar	Normalizar	Calcular el máximo y el mínimo: 30 y 80. Calcular x' como $x - 30 / 50$.	Calcular x' como $x - 30 / 50$.
Balancear	Oversampling	Aplicar sobremuestreo	-

Flujo de preparación de datos

Split del dataset

Calcular y aplicar las transformaciones





Data drift. Ejemplo 1: codificación.

Split del dataset

Calcular y aplicar las transformaciones

TRAIN dataset

A	B	C
	PERRO	
	PERRO	
	GATO	
	PERRO	

Identifico codificación:
OHE, dos categorías,
B_PERRO y B_GATO

Aplicar codificación

TRAIN transformado

A	B_PERRO	B_GATO	C
	1	0	
	1	0	
	0	1	
	1	0	

TEST dataset

A	B	C
	PERRO	
	GATO	
	PATO	

Aplicar codificación



Posibles soluciones:

- Ignorar la categoría desconocida.
- Re-analizar los datos/el problema.

TEST transformado

A	B_PERRO	B_GATO	C
	1	0	
	0	1	
	0	0	

Para una determinada variable, en el dataset de Test hay una categoría que no está en el dataset de Train.



Data drift. Ejemplo 2: escalamiento.

Split del dataset

Calcular y aplicar las transformaciones

TRAIN dataset

A	B	C
	10	
	20	
	5	
	50	

Normalización Min-Max:
Min = 5, Máx = 50
→ $(x - 5)/(50 - 5)$

Aplicar normalización

TRAIN transformado

A	B	C
	0,11	
	0,33	
	0	
	1	

TEST dataset

A	B	C
	15	
	2	
	55	

Aplicar normalización



Posibles soluciones:

- En la mayoría de los casos, esto no es un problema.
- Se puede hacer un *clipping* (forzar los valores a que caigan dentro del intervalo).

TEST transformado

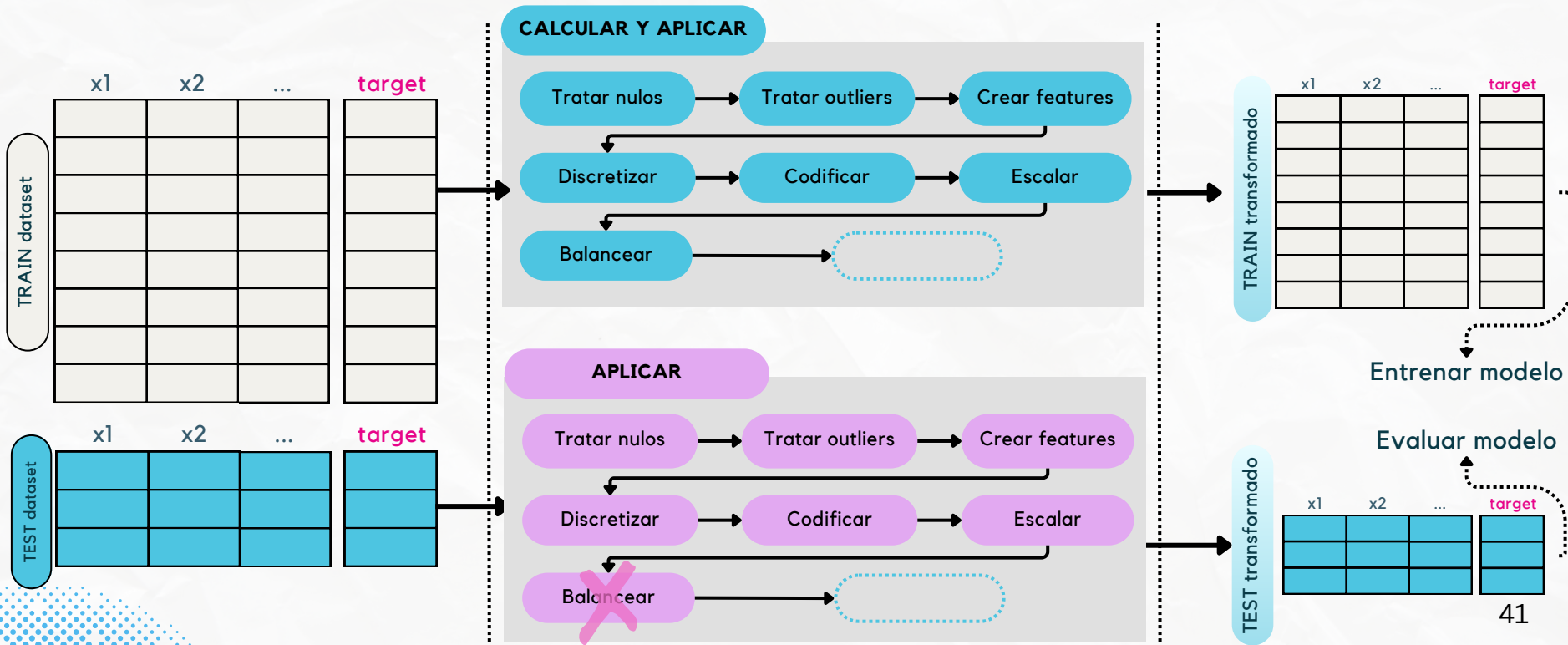
A	B	C
	0,22	
	-0,06	
	1,11	

En Test una cierta variable toma un valor más grande que el máximo (y/o más chico que el mínimo) de esa variable en Train.

Flujo de preparación de datos

Split del dataset

Calcular y aplicar las transformaciones



¿Te quedaron dudas?



- Recordá que podés consultarnos durante la clase preniendo la cámara y hablando, o simplemente escribiendo en el chat.
- También podés escribirnos por correo las veces que sea necesario (por favor incluir a ambas docentes).



Esp. Lic. María Carina Roldán
macroldan@fi.uba.ar



Esp. Ing. Ariadna Garmendia
arigarmendia@gmail.com

Y si tenés un feedback, un pedido, una sugerencia, etc. y no te animás a decirlo, podés dejarlo por escrito en la encuesta voluntaria y anónima (disponible durante todo el bimestre).